

Implementation of a Network Security Device Using Raspberry Pi Firewall

CSL5_OSY Operating Systems

Student ID: 4228665



London South Bank
University

Submission Date: 9th December 2025

Abstract

This abstract shows the Work-In-Progress (WIP) implementation and preliminary evaluation of a cost-effective, dedicated Network Security Device utilising a Raspberry Pi single-board computer (SBC). Driven by the coursework brief and the rising cost of commercial firewall devices. This coursework uses open-source software (Ubuntu's iptables) to transform the SBC into a powerful, low-footprint packet-filtering, network-wide firewall. The implementation focuses on establishing secure network segmentation using multiple firewall zones and policies to mitigate internal and external threats, addressing the project requirement for a dedicated security solution.

Contents

0	Introduction	3
1	Domain of Knowledge	5
1.1	What is a Firewall?	5
1.2	Firewall OS Integration and Install	6
1.3	System Requirements and Design	6
2	Planning and Implementation	7
2.1	Initial Setup and OS install	7

0 Introduction

Network security has always been a warranted topic of concern in computer networks. A way to prevent or mitigate some of that concern on a network is to implement a firewall. Firewalls are able to prevent the spread of unwanted or malicious programs or data across networks/zones (Blansit 2009) and reduce the damage of a compromised network. While enterprise-level firewalls can cost upwards of \$40,000 USD (Blue Sky Systems 2025), a local/home firewall can be made inexpensive using a Raspberry Pi single-board computer (SBC) and open source software for significantly less (The Pi Hut 2025) (Deciso B.V. 2025). This project implements a lightweight firewall solution using a Raspberry Pi, turning the SBC into a dedicated network security device with multiple firewall zones and policies to ensure network traffic traverses the network as per the example in figure 1. This will ensure proper packet filtering and traffic management within the network the firewall is deployed.

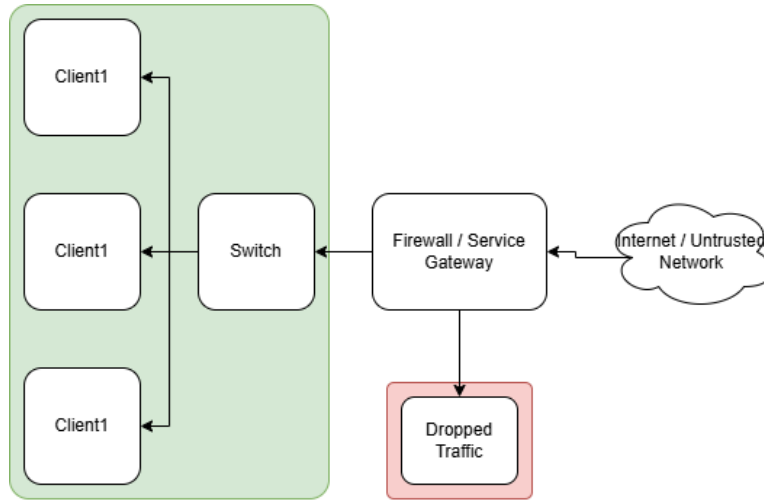


Figure 1: A diagram showcasing basic firewall filtering. showing how incoming traffic comes from the internet or an untrusted network, connects to the Firewall / Service gateway and is either passed to the LAN or dropped.

At its core, the coursework task put out is to install and configure a firewall application on a Raspberry Pi to act as a standalone firewall, as previously mentioned, while showing adequate knowledge and understanding of networking protocols (via traffic control, packet filtering, routing) and other approaches with self-hosting network devices, such as network monitoring and general observability (OpenCloudification 2025) (Dadiyala et al. 2023). By using the Pi's onboard hardware and an additional network interface card (NIC), you can secure a home or small-to-medium-sized office network when paired with a Linux-based operating system like OPNsense (Deciso B.V. 2025) (an open-source firewall and routing platform). OPNsense resembles a VM-based option for a service gateway (similar to Juniper's offering on its SRX line of hardware) that can provide routing, multi-WAN support, and VPNs, as well as firewall-specific features like network filtering, rule-based forwarding, and traffic (deep packet) inspection. What makes it more than just a firewall (a service gateway) is that it operates at all layers of the OSI model, not just layers 1-4. This project creates a general network service gateway capable of managing traffic between three distinct network interfaces, each representing different security zones.

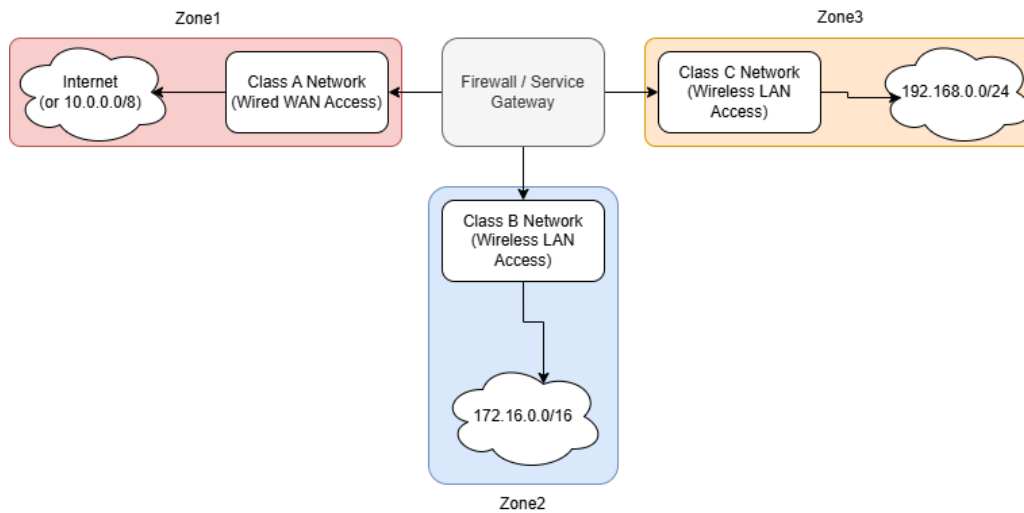


Figure 2: A diagram showing the different network ranges, the interfaces (and types) they're connected to and how they'll be isolated and sat within different firewall zones.

TO EXPLAIN DIFFERENCES BETWEEN CLASSFUL NETWORK DECLARATIONS

This task calls for building a multi-interface firewall supporting Class-A, Class-B, and Class-C networks (as highlighted in Figure 2), and implementing firewall policies including default-deny rules and essential service permissions (allowing DNS, DHCP, and ICMP traffic) between the zones. Moreover, the work should allow internal/management network communication (to access the firewall), network address translation for internet access, and some form of logging for analysis and troubleshooting. Furthermore, the project calls for performance monitoring and evaluation to assess the system's scalability and effectiveness under various traffic conditions outlined in the brief.

1 Domain of Knowledge

1.1 What is a Firewall?

A firewall is a network security device (or virtual application) that monitors and controls traffic based on rules and policies defined by an administrator (Cisco 2024). As shown in figure 1, firewalls typically sit on the network edge, between the local network and a wider WAN (in this case, the internet via the class A interface) to filter / outright block untrusted traffic. Despite sitting on the edge of a network, firewalls also filter and allow/block traffic internally (on the LAN side) for trusted communication between networks (as shown in the ranges in Figure 2).

Firewalls are mainly used in networks for packet filtering, which requires inspecting IP packets and checking headers of the packets to determine their legitimacy against pre-defined firewall policies and rules (Palo Alto Networks 2025a). A firewall policy (or rule) is a declarative line of code that specifies whether a packet can traverse across the firewall or not, depending on specific attributes. For instance if a ubuntu server were only to allow incoming ssh connections and not telnet (as it is insecure and unencrypted), you would configure a UFW (Ubuntu's 'Uncomplicated Firewall') policy that either allows (whitelisting) only traffic on port 22 (SSH default port) or one that disallows (blacklisting) port 23 for both IPv4 and IPv6. This policy is shown in figure 3. Firewalls also filter based on other attributes of packets, across the OSI model, taking into account the source and destination of a packet and the protocol of the packet, all of which can determine whether a packet is allowed, dropped, or denied by a firewall.

```
s4228665@control11-u:~$ sudo ufw status
Status: active
```

To	Action	From
22/tcp	ALLOW	Anywhere
23	DENY	Anywhere
22/tcp (v6)	ALLOW	Anywhere (v6)
23 (v6)	DENY	Anywhere (v6)

```
s4228665@control11-u:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

To	Action	From
22/tcp	ALLOW IN	Anywhere
23	DENY IN	Anywhere
22/tcp (v6)	ALLOW IN	Anywhere (v6)
23 (v6)	DENY IN	Anywhere (v6)

```
s4228665@control11-u:~$
```

Figure 3: A screenshot from a Raspberry Pi running Ubuntu showcasing UFW policies to block port 23 (Telnet) and allow port 22 (SSH) connections. The verbose output shows the default rules of allowing all outgoing traffic but disabling default traffic going to the host.

To further secure a network, designers often segment it into different zones (Palo Alto Networks 2025c), each with its own trust levels and isolated from the others via firewall policies that allow only necessary traffic between hosts. The coursework task also calls for this via an external Class-A network representing the untrusted internet connection, and two internal networks (Class-B and Class-C) simulating different frequency bands with different security requirements.

Following the "Never trust, always verify" approach (Ravi et al. 2025) to network security, firewalls should follow the principle of least privilege (PoLP) (Palo Alto Networks 2025b). PoLP works by limiting the amount of accessible data, resources, and applications to hosts/roles explicitly allowed to access them. Figure 3 does this by explicitly allowing only SSH connections instead of working and allowing SFTP (as an example) connections, which would be allowed if there were no firewall in place. This approach minimises the potential attack surface on the network and ensures that only necessary communications are allowed.

1.2 Firewall OS Integration and Install

To install an operating system on a typical (x86-based CPU) pc you would create a boot medium. This can be done by installing the OS' image (.iso) file then running it through software that would take the .iso file and create a bootable usb-drive (or disk) that when inserted into the PC (and a user navigates to the boot options of the BIOS/UEFI) will boot to the install medium so a user can configure the OS install. This is how OPNsense would be installed on a supported x86 system. Unlike traditional x86-based operating systems (in this case a firewall distribution) that require installation to a storage medium, ARM-based systems like the Raspberry Pi require a different process. Instead of mounting the ISO image for installation, the Raspberry Pi requires a firmware image to be directly written (or "flashed") to the SD card, which serves as both the boot medium and primary (main) storage component.

In discovering this it was important to change plans to pick an implementation that would support the ARM-based cpu that the Raspberry Pi supports. One implementation of a network firewall is iptables, Ubuntu's command line interface to Linux's netfilter kernel module (Meysam 2020). This approach gives several advantages over the x86 OPNsense, mainly around demonstrating understanding of how the network modules of the Linux kernel work.

Ubuntu's netfilter framework, accessed through iptables, has features to meet the coursework requirements as it includes stateful packet inspection, network address translation (NAT), connection tracking, and logging (GeeksforGeeks 2025). The system can be configured as a dedicated network gateway by enabling IP forwarding, configuring the three network interfaces (one USB-to-Ethernet adapter for the Class-A WAN interface, the built-in Ethernet for Class-B, and the onboard WiFi for Class-C as per figure

1.3 System Requirements and Design

2 Planning and Implementation

2.1 Initial Setup and OS install

To build the firewall, Ubuntu 24.04 LTS needs to be installed. As mentioned in an earlier section, ARM-based CPUs need to have their install media flashed to their storage device. The Raspberry Pi imager allows users to do this easily, as per the figure 8 BASE IMAGE). The screenshot shows the Raspberry Pi model provided (a Pi 4 B), the OS to install (Ubuntu 24.04 LTS), and the storage medium (an SD card before using the provided one).

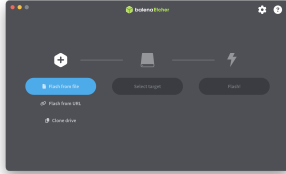


Figure 4: Balena etcher ready to flash the SD card.

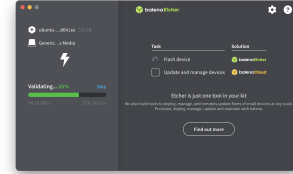


Figure 5: Balena etcher validating the OS flash.

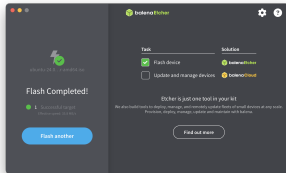


Figure 6: Another

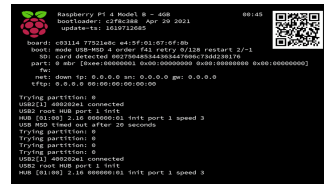


Figure 7: Another

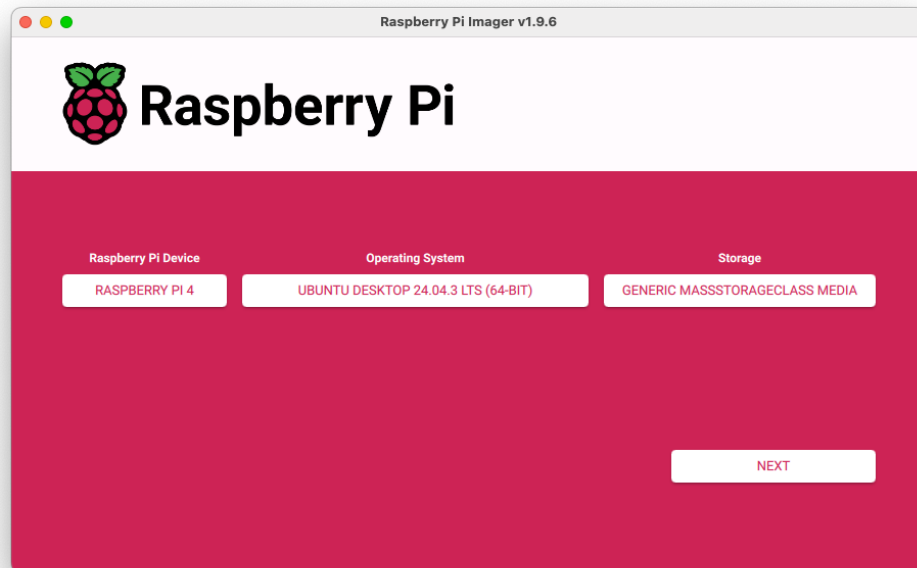


Figure 8: A screenshot from the Raspberry Pi imager, a software that allows users to easily pick and configure a Pi to run headlessly without any configuration needed on the device to get it up and running.

From this point, the imager will begin to download the OS and flash the SD card with the new OS, and once complete, it will show as figure 9.

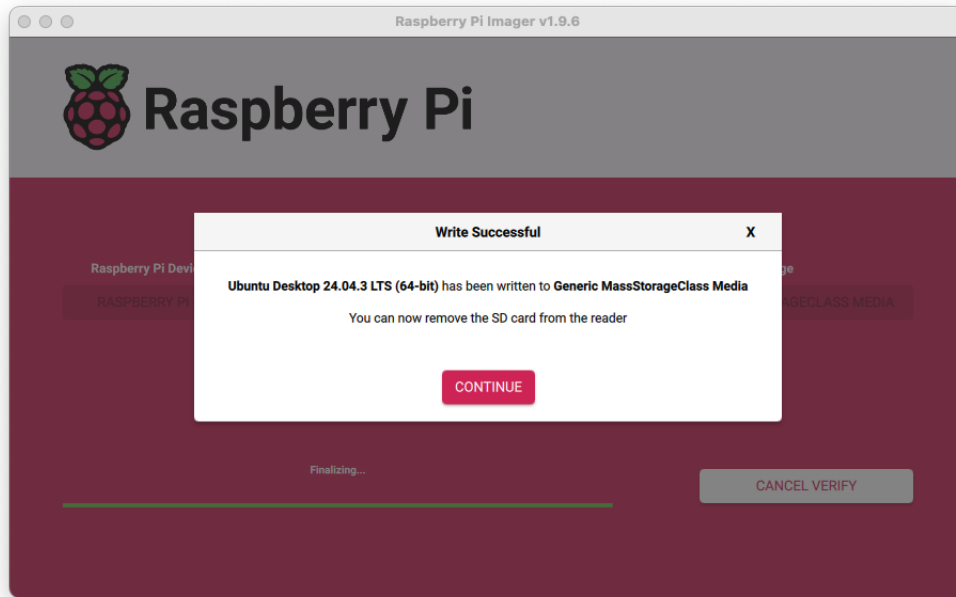


Figure 9: A screenshot from the Raspberry Pi imager, after installing ubuntu 24.04 LTS.

References

- Blansit, B. Douglas (2009). “Firewalls: Basic Principles and Some Implications”. In: *Journal of Electronic Resources in Medical Libraries* 6.3, pp. 260–269. DOI: 10.1080/15424060903167377.
- Blue Sky Systems (2025). *Palo Alto Networks PA-3400 Series PA-3420 Security Appliance*. Accessed: 2025-11-08. URL: <https://shop.blueskysystems.co.uk/product/Networking/Firewalls/Palo-Alto-Networks/PAN-PA-3420-Palo-Alto-Networks-PA-3400-Series-PA-3420---Securi?prodid=4273258>.
- Cisco (Aug. 2024). *What Is a Firewall?* Cisco Systems. URL: <https://www.cisco.com/site/uk/en/learn/topics/security/what-is-a-firewall.html> (visited on 11/09/2025).
- Dadiyala, Charanjeet et al. (July 2023). “Designing and Implementing an Effective Cybersecurity Home Lab for Detection and Monitoring”. In: *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. DOI: 10.1109/ICCCNT56998.2023.10306937.
- Deciso B.V. (2025). *OPNsense: Open Source Firewall and Routing Platform*. Accessed: 2025-11-08. URL: <https://opnsense.org/>.
- GeeksforGeeks (2025). *iptables command in Linux with Examples*. GeeksforGeeks. URL: <https://www.geeksforgeeks.org/linux-unix/iptables-command-in-linux-with-examples/> (visited on 11/11/2025).
- Meysam (2020). *What Is iptables and How to Use It?* Meysam. URL: <https://medium.com/skilluped/what-is-iptables-and-how-to-use-it-781818422e52> (visited on 11/09/2025).
- OpenCloudification (2025). *Scalable Officelab Homelab Monitoring*. Accessed: 2025-11-08. URL: https://www.opencloudification.com/wp-content/uploads/2025/07/scalable_officelab_homelab.pdf.
- Palo Alto Networks (2025a). *What Is a Packet Filtering Firewall?* Palo Alto Networks. URL: <https://www.paloaltonetworks.co.uk/cyberpedia/what-is-a-packet-filtering-firewall> (visited on 11/09/2025).
- (2025b). *What Is Network Segmentation?* Palo Alto Networks. URL: <https://www.paloaltonetworks.com/cyberpedia/what-is-network-segmentation> (visited on 11/09/2025).
- (2025c). *What Is the Principle of Least Privilege?* Palo Alto Networks. URL: <https://www.paloaltonetworks.co.uk/cyberpedia/what-is-the-principle-of-least-privilege> (visited on 11/09/2025).
- Ravi, C. et al. (2025). “Beyond the Firewall: Implementing Zero Trust with Network Microsegmentation”. In: *Nanotechnology Perceptions* 21, pp. 560–578.
- The Pi Hut (2025). *Raspberry Pi 5*. Accessed: 2025-11-08. URL: <https://thepihut.com/products/raspberry-pi-5>.